

MÓDULO 1
**Seguridad Informática
y Criptografía**
Ciberseguridad

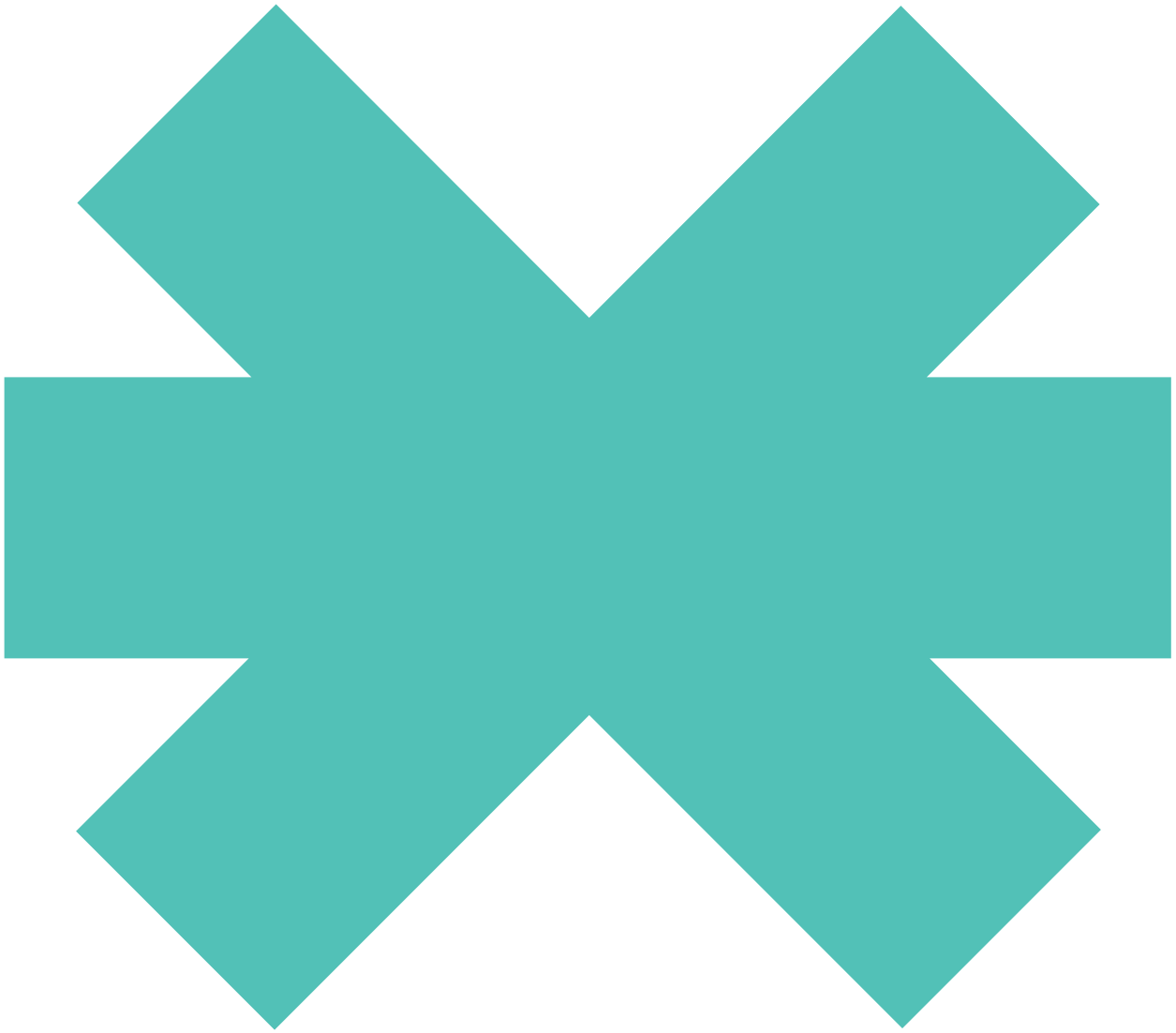
4

Cifrado simétrico



New
Technology
School

Tokio.



4 Cifrado simétrico

Sumario

4.1	Principales métodos del cifrado simétrico
-----	---

60

4.1 Principales métodos del cifrado simétrico

El cifrado simétrico consiste en **utilizar la misma clave de cifrado tanto para el emisor como para el receptor**. En este sentido, ambos han de consensuar la clave que se va a usar. Existen varios métodos de cifrado simétrico que describiremos a lo largo de este tema.

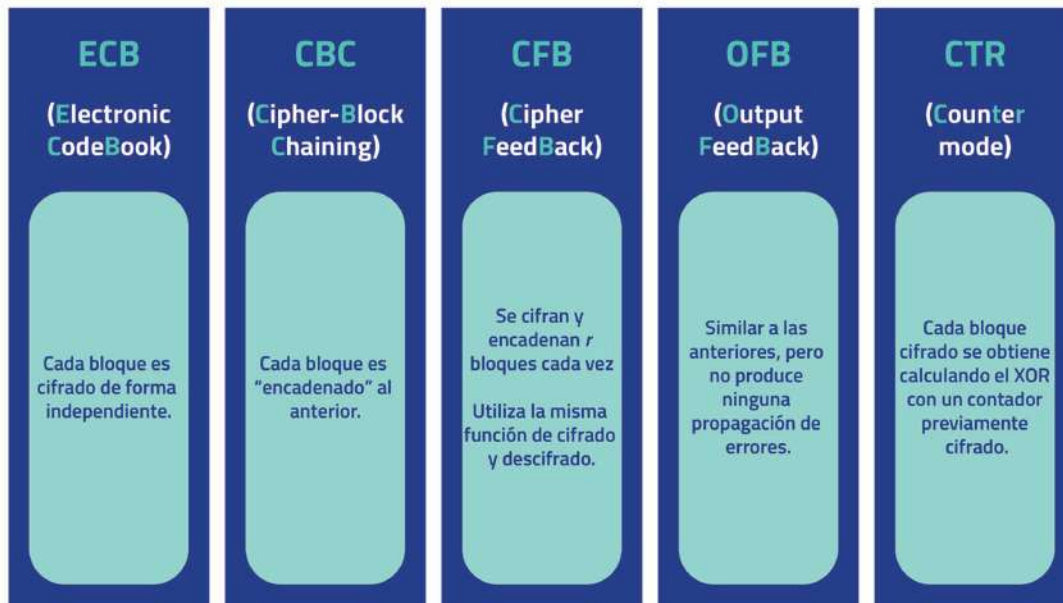


Figura 32

Modo ECB

El modo ECB (*Electronic CodeBook*) **opera estrictamente sobre bloques**, esto quiere decir que es un sistema diseñado para la encriptación de una serie de bloques de datos. Cuando la información que se ha de cifrar no posee exactamente la longitud múltiplo del tamaño del bloque definido, se rellenará automáticamente con un **padding** o con un relleno definido.

Este sistema primero divide la información a encriptar en bloques de un tamaño determinado de manera que cada uno se codifique luego por separado utilizando la misma clave simétrica tanto para encriptar como para descifrar la información.



Figura 33

Presenta los siguientes **inconvenientes**:

- No oculta los bloques.
- Un atacante puede modificar el orden de los bloques, así como insertar otros bloques o eliminarlos.

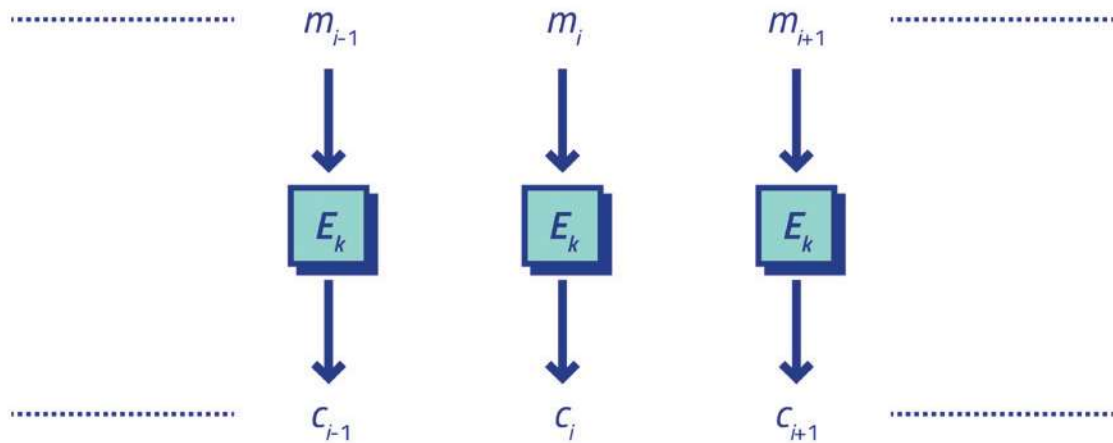
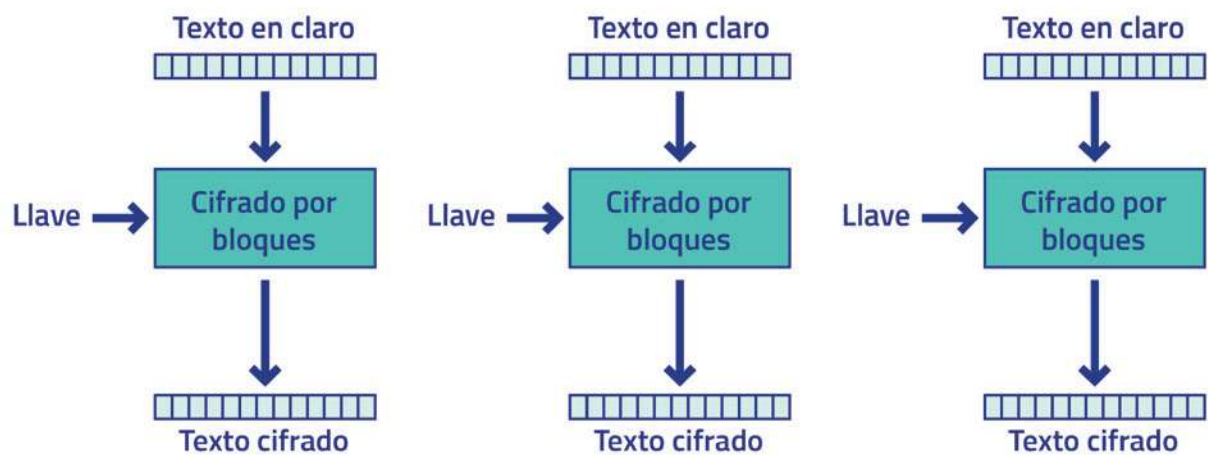


Figura 34

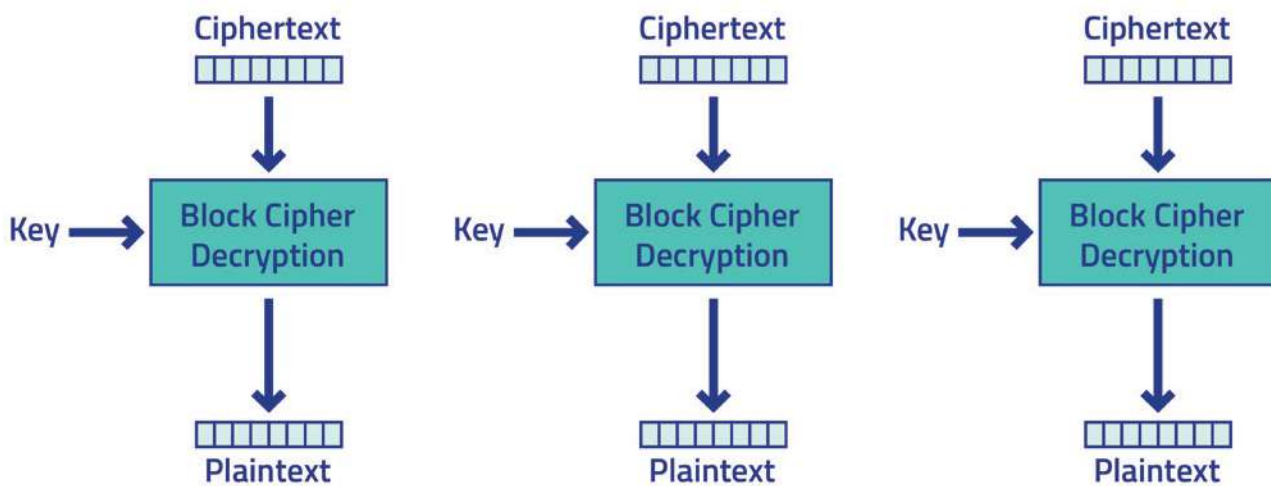
En la siguiente imagen, se observa claramente el **flujo del modo ECB**:



Cifrado en modo de operación Electronic Codebook (ECB)

Figura 35

Para descifrar el texto:



Electronic Codebook (ECB) mode decryption

Figura 36

De esta forma, se pueden extraer **patrones de texto** que pueden revelar el contenido del fichero cifrado:

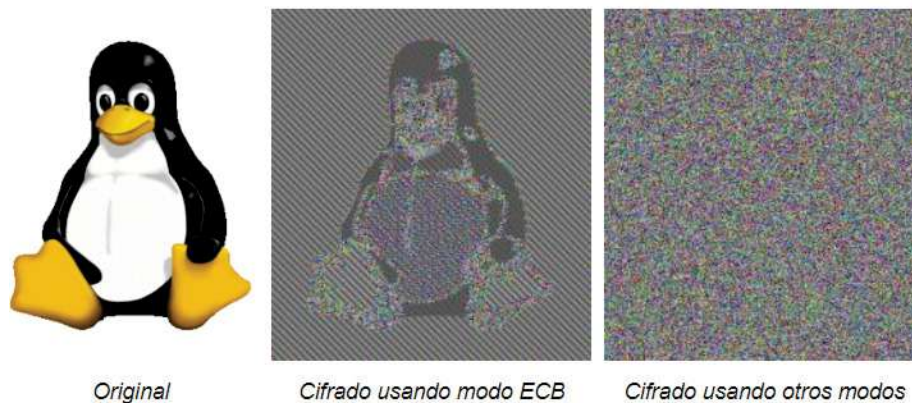


Figura 37

Ejercicio de reflexión

Crear un JAR en Java capaz de llevar a cabo un cifrado y descifrado de este tipo por medio de dos parámetros: la clave y el texto que se va a cifrar.

Ejercicio de reflexión

Buscar cualquier imagen en formato BMP y descargarla. A continuación:

- Cifrarla utilizando el modo ECB. Para ello, podemos utilizar una línea similar a la siguiente:

```
# openssl enc -aes-128-ecb -in <imagen.bmp> -out <imagen_cifrada.bmp> o bien el JAR anteriormente desarrollado.
```

- Ahora es necesario restaurar la cabecera de la imagen, que también ha sido cifrada durante el proceso. Para ello, ejecutamos la siguiente orden:

```
# dd if=index.bmp of=index_cifrada.bmp bs=1 count=54 conv=notrunc
```

- ¿Qué observamos? ¿Por qué ocurre esto? ¿Cuál es la conclusión?

VECTOR DE INICIALIZACIÓN (IV)

Un vector de inicialización es un bloque de *bits* necesario para realizar un cifrado de flujo o un cifrado de bloque, la función que realiza es la de encriptar el primer bloque de texto plano del mensaje.

El IV presenta las siguientes características:

- No es necesario que sea secreto, pero sí impredecible.
- Nunca se deben usar varios IV con la misma clave.
- Solución: guardar el IV utilizado al comienzo del fichero.
- Mínimo 16 bytes = 128 bits = 2^{128} posibles valores.
- Resulta aconsejable generar de forma aleatoria, para cada proceso de cifrado, el fichero, el mensaje, etc.

Modo CBC

Este modo de cifrado (*Cipher Block Chaining*) se diferencia del anterior en que **el bloque de cifrado es absolutamente dependiente del anterior**, lo que dificulta la obtención de patrones. Este sistema realiza una operación XOR entre un bloque de texto plano y el bloque de texto cifrado anterior, además presenta un problema, y es que no permite realizar cambios rápidos sobre la información cifrada, el proceso debería de repetirse por completo para cada modificación.

Uso correcto del modo CBC:

- Generar un IV aleatorio.
- Cifrar en modo CBC.
- Concatenar el IV con la salida antes de guardar en disco.

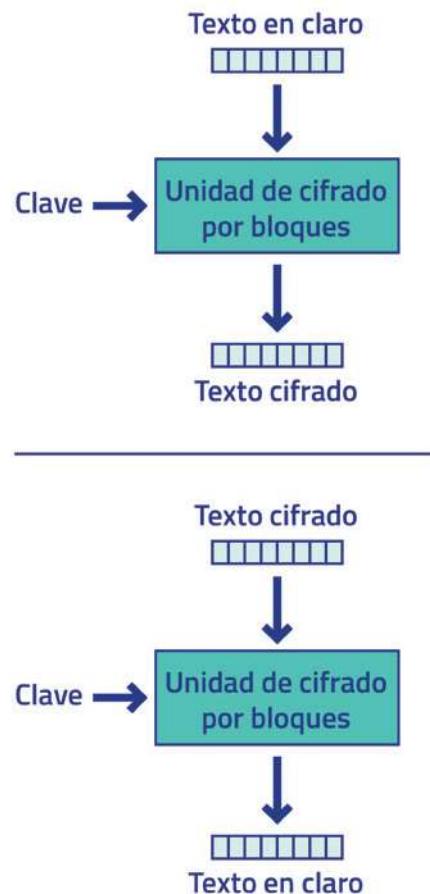


Figura 38

Modo OFB

En el modo de OFB (*Output Feedback*), se utiliza la **clave secreta** para crear un **bloque pseudoaleatorio** al que se le aplica la **operación XOR con el texto en claro** para generar el texto cifrado. En este caso, también se necesita un **vector de inicialización** que ha de ser **único para cada mensaje cifrado**, pues de lo contrario, se comprometería la seguridad del sistema. Tampoco se puede paralelizar.

El funcionamiento del modo OFB es el siguiente:

- Generar un IV aleatorio que se encripta con la clave de cifrado del algoritmo de bloques.
- Realizar la operación XOR para generar el texto cifrado.
- El vector encriptado del primer punto se emplea como segundo IV para el siguiente bloque a encriptar.
- Se repite el proceso las veces necesarias para encriptar todos los bloques.

Modo CFB

El modo CFB (*Cipher Feedback*) es muy similar al anterior, se emplea para generar una secuencia de bloques clave. Transforma un cifrado de bloque en un cifrado de flujo de sincronización automática, los cambios de texto sin formato se distribuyen siempre en el texto cifrado y no se puede paralelizar.

El funcionamiento del modo CFB es el siguiente:

- Generar un IV aleatorio que se encripta con la clave de cifrado del algoritmo de bloques.
- Con el resultado del paso anterior se realiza una operación XOR sobre el bloque de texto plano que se va a encriptar.
- Se reorganiza la cadena para obtener un nuevo IV que se vuelve a encriptar con algoritmos de cifrado de bloques.
- Se repite el proceso las veces necesarias para encriptar todos los bloques.

Modo CTR contador

Al igual que el OFB, el cifrado de bloques constituye, esencialmente, un **cifrado de flujo**, pues cifra valores sucesivos de un contador. Puede actuar como **contador** cualquier **función sencilla** que produzca una **secuencia de números** y cuyos resultados se repitan con una frecuencia muy reducida.

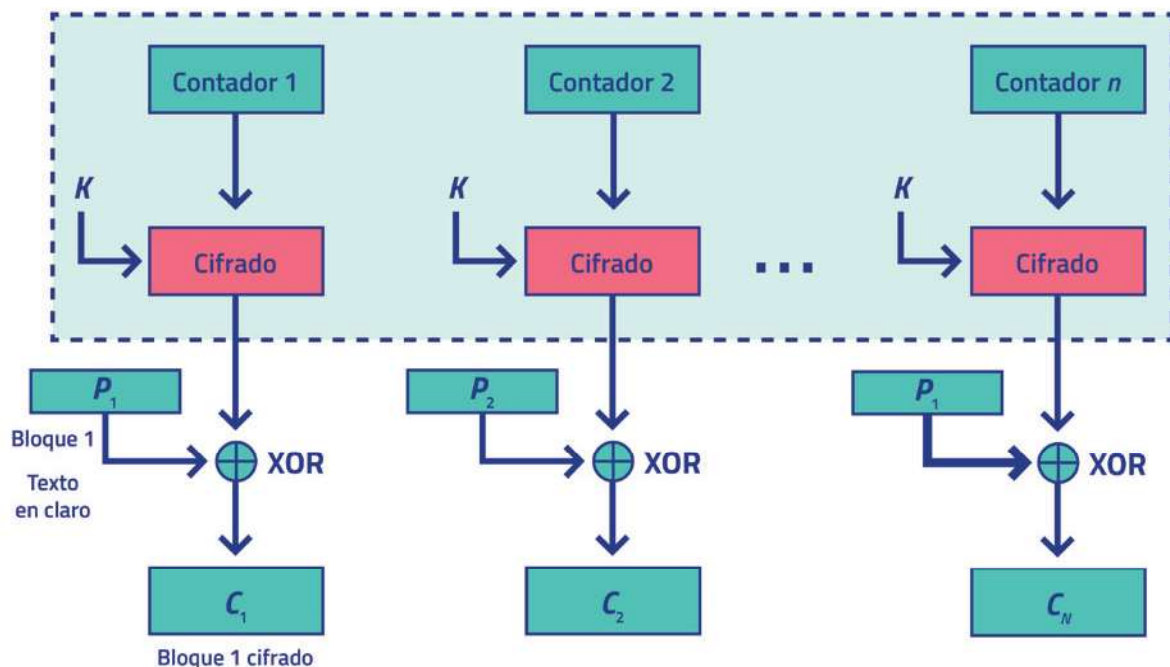


Figura 39

Modos autenticados

Consisten en **añadir un hash al final del texto cifrado** (un hash es un algoritmo matemático que transforma un bloque de datos en una nueva serie de caracteres con una longitud fija) con el objeto de garantizar la integridad del mensaje:

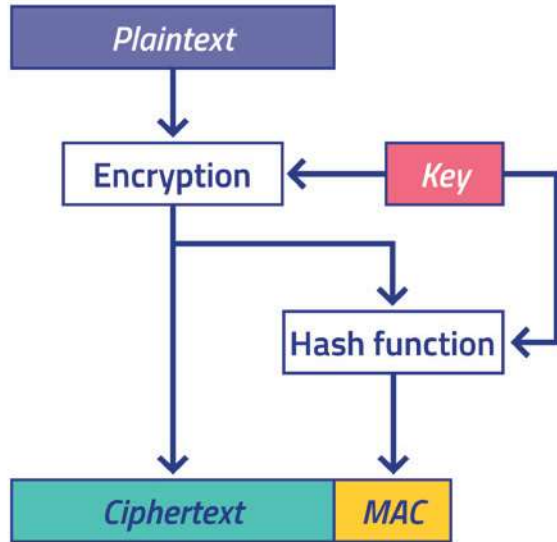


Figura 40

En la siguiente tabla, se muestra un resumen de los modos que hemos descrito:

	ECB	CBC	CTR	
			CCM	GCM
Seguridad semántica	✗	✓	✓	✓
Encadenamiento	✗	✓	✓	✓
Propagación errores	✓ Limitada	✓ en descifrado ✗ en cifrado	¡Esencial NO repetir el IV para una misma clave!	
Recuperación de errores	-	2 bloques		

Figura 41

Los modos autenticados (GCM, CCM) añaden seguridad adicional, pero **resulta esencial no repetir IV**. Si esto no se puede garantizar, es preferible utilizar CBC (que tampoco debería repetir IV, si bien no es tan sensible).

Cifrados DES

Se trata de un **cifrado simétrico** que fue seleccionado como **estándar del FIPS** en **1976**, extendiéndose ampliamente por todo el mundo. En concreto, este algoritmo nació en **1974**, después de que la NBS hubiese señalado, en 1973, la **necesidad de contar con un algoritmo de cifrado para la protección de datos durante su transmisión y almacenamiento**. En consecuencia, IBM presentó su propuesta de algoritmo, al que la NSA impuso una longitud de **56 bits**, lo que significa $2^{56} = 7,2 \times 10^{16}$ claves.

Este modo se basa en un **cifrado en bloque**, tal y como se muestra en la imagen:

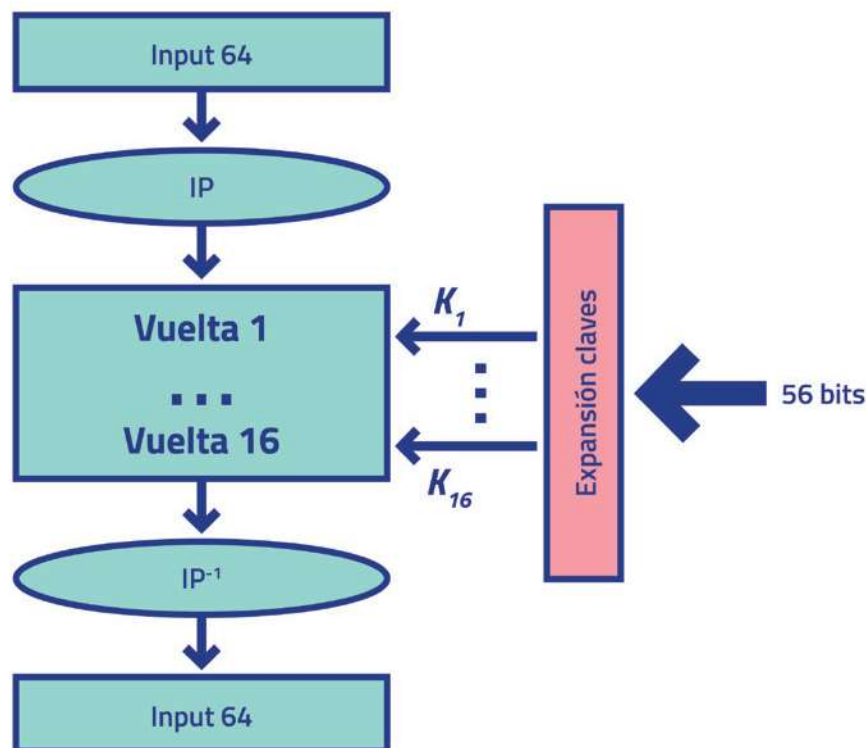


Figura 42

Su **estructura** es la siguiente:

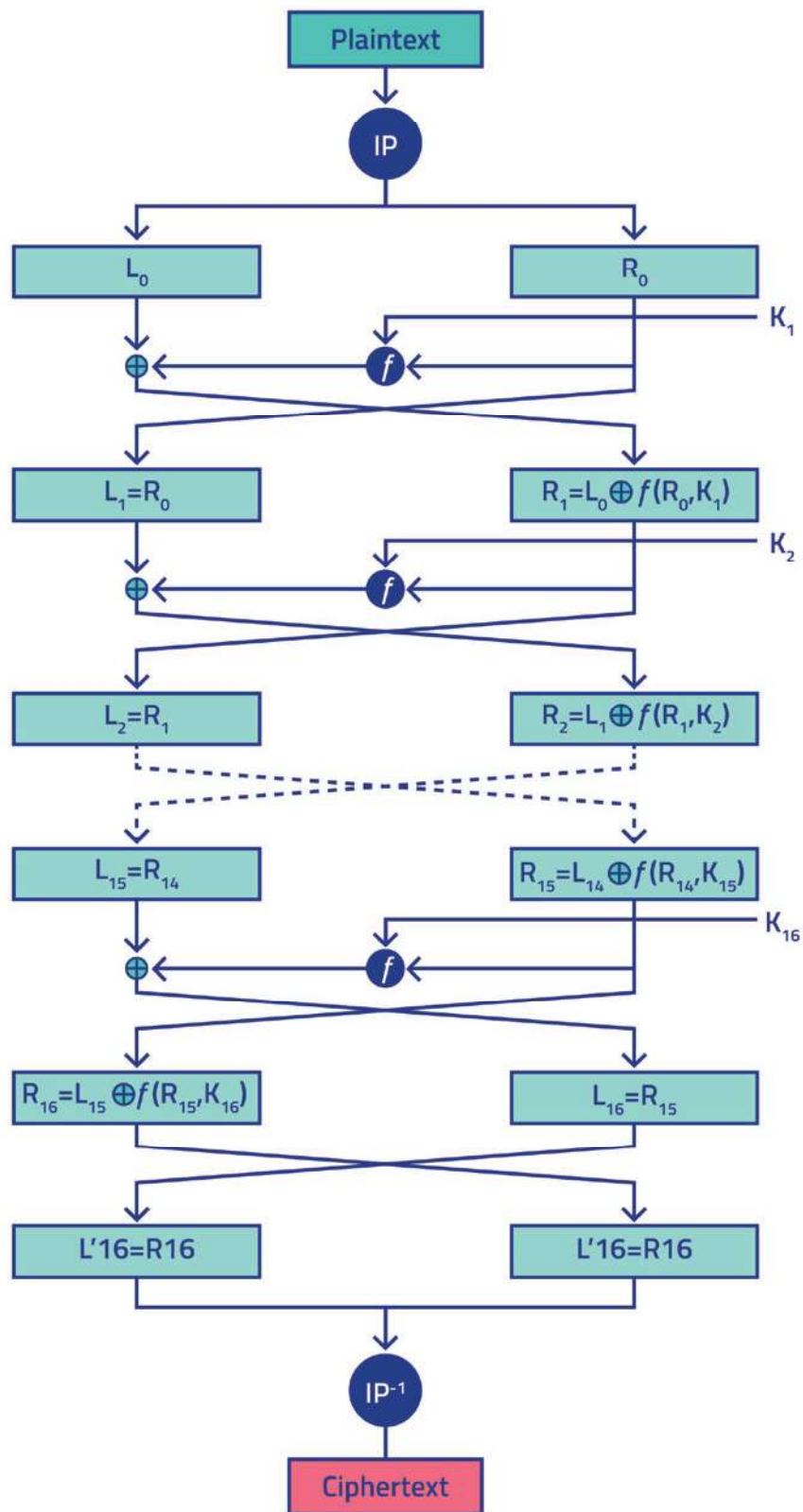


Figura 43

Cifrado TDES

Actualmente, el hermano pequeño de triple DES. DES es un algoritmo **extremadamente inseguro** y, por esta razón, se decidió **concatenar varias interacciones**, ya que para las compañías, resulta demasiado caro actualizar tanto el *software* como el *hardware*.

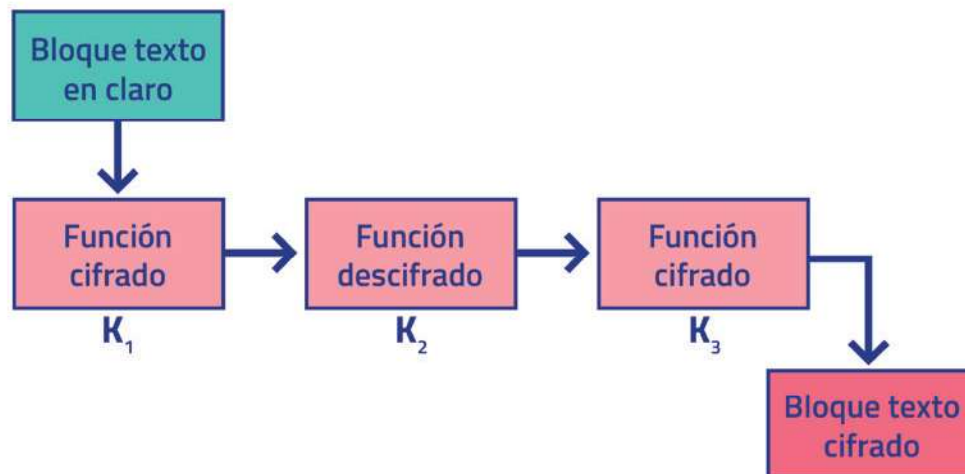


Figura 44

Con todo, la actualización de este algoritmo tampoco es recomendable; de hecho, compañías como Redsys ya están dejando de utilizarlo. Ciertamente, hay mucho trabajo por delante, pues no es sencillo forzar a tantas compañías a que actualicen el algoritmo, lo que implica intercambio de claves, actualizaciones de *software*, etc.

Cifrado AES

El cifrado AES, también conocido como *Advanced Encryption Standard* o Rijndael, es un **esquema de cifrado por bloques** adoptado como **estándar** por el **Gobierno de Estados Unidos**. A comienzos del siglo XXI, concretamente en el año **2001**, se estableció como estándar efectivo.

Previamente, en 1997, el NIST había decidido realizar otro concurso para optimizar la seguridad que, en ese momento, el cifrado DES no podía ofrecer y, tras varios años de estudio por parte de distintos especialistas en la materia, se seleccionó el cifrado AES.

La convocatoria del NIST especificaba que todos los algoritmos presentados debían reunir los siguientes **requisitos**:

- Ser de dominio público, es decir, disponibles para todo el mundo.
- Ser de cifrado simétrico, además de soportar bloques de, como mínimo, 128 bits.
- Presentar claves de cifrado de 128, 192 o 256 bits.
- Ser implementables tanto en *software* como en *hardware*.
- Presentar un diseño eficiente en microprocesadores de cualquier ancho de palabra, a partir de 8 bits, usados en tarjetas inteligentes.

AES Design

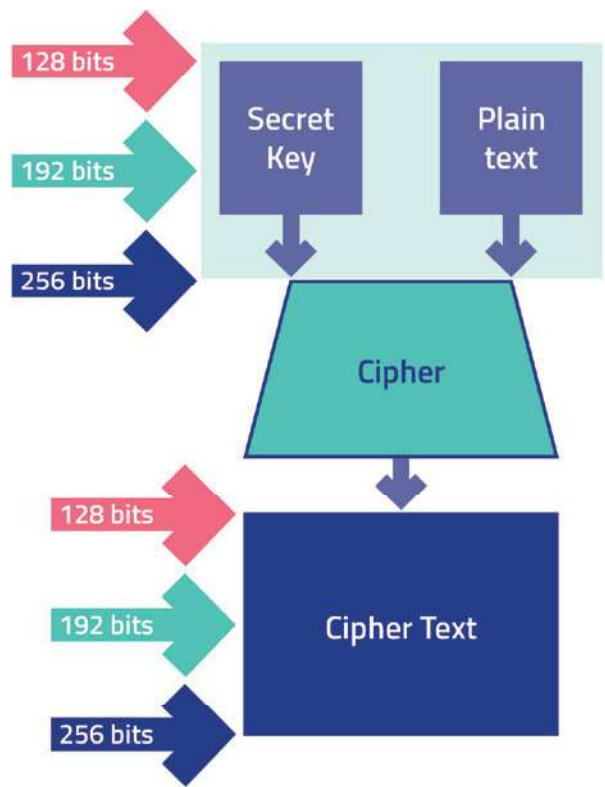


Figura 45

Los **ataques por fuerza bruta** conllevan las siguientes **repercusiones**:

Longitud de la clave	Tiempo necesario para romper la clave
40 bits	2 segundos
48 bits	9 minutos
56 bits	40 horas
64 bits	14 meses
72 bits	305 años
80 bits	78.250 años
96 bits	5.127.160.311 años
112 bits	336.013.578.167.538 años
128 bits	22.020.985.858.787.784.059 años
192 bits	$1.872 \cdot 10^{37}$ años
256 bits	$9.1 \cdot 10^{50}$ años

Figura 46